

ARC-AGI-1: Estudo Comparativo entre Abordagem Baseline e Program Synthesis via CEGIS com Feedback Semântico

Fabio Luiz da Costa Cieslak*, Izadora Candotti de Oliveira*, Matheus Machado*

Repositório GitHub: [Jubarte27/arc-agi-1](https://github.com/Jubarte27/arc-agi-1)
<https://github.com/Jubarte27/arc-agi-1>

Abstract

Este artigo apresenta uma análise resumida do repositório `arc-agi-1`, que realiza uma avaliação comparativa de abordagens de síntese de programas (Program Synthesis) no benchmark ARC-AGI-1. Investigamos duas metodologias principais utilizando modelos de linguagem de grande porte (LLMs) via APIs do Google Gemini (usando o modelo `gemini-3.1-flash-lite` por padrão) e Groq: a abordagem Baseline (1-Shot) e a abordagem baseada em Síntese Indutiva Guiada por Contraexemplos (CEGIS) com feedback semântico. Além disso, o repositório destaca-se pela implementação de proteções robustas para contas de nível gratuito (Free Tier Protections), garantindo a resiliência dos experimentos frente a limitações de taxa (RPM/TPM), cotas diárias e falhas de rede.

Código — <https://github.com/Jubarte27/arc-agi-1>

1 Introdução

O benchmark Abstraction and Reasoning Corpus (ARC-AGI) (Chollet 2019) propõe o desafio de desenvolver sistemas de inteligência artificial capazes de adquirir novas habilidades de forma rápida e generalizável, simulando o raciocínio indutivo humano. No contexto de síntese de programas (Program Synthesis), o objetivo consiste em gerar um algoritmo capaz de mapear grades de entrada (grids) para suas respectivas grades de saída com base em poucos exemplos demonstrativos de treino.

O repositório `arc-agi-1` (da Costa Cieslak, de Oliveira, and Machado 2026) fornece um framework robusto para comparar duas estratégias distintas de síntese de programas utilizando modelos de linguagem avançados, com suporte nativo a restrições de APIs comerciais gratuitas.

2 Metodologia

O framework compara duas abordagens principais para a resolução das tarefas do ARC-AGI-1:

*These authors contributed equally.

2.1 Abordagem Baseline (1-Shot)

Na estratégia Baseline (1-Shot), o modelo de linguagem de grande porte recebe os pares de demonstração de treinamento de uma determinada tarefa diretamente no prompt. O modelo deve escrever diretamente uma função Python autônoma chamada `transform(grid)`. Essa função é executada uma única vez e avaliada tanto no conjunto de demonstrações de treinamento quanto na grade de teste oculta. Para que a tarefa seja considerada resolvida com sucesso, a função gerada deve passar em ambos os conjuntos (treino e teste).

2.2 Abordagem CEGIS

A abordagem CEGIS (*Counterexample-Guided Inductive Synthesis* com Feedback Semântico) introduz um ciclo iterativo de refinamento de código. Inicialmente, o modelo gera um programa preliminar. Caso este programa falhe em qualquer uma das demonstrações de treinamento, o sistema captura um contraexemplo semântico contendo:

- A grade de entrada correspondente;
- A saída esperada pela tarefa;
- A saída realmente gerada ou o erro de execução em tempo de execução.

Este contraexemplo é formatado e realimentado no contexto do chat, solicitando ao modelo a correção do código. Esse loop de feedback se repete até que o programa passe em todas as demonstrações ou atinja o limite máximo de iterações configurado pelo parâmetro `MAX_CEGIS_ITERS` (padrão de 5 iterações). Após a convergência ou esgotamento de tentativas, o programa final é avaliado no teste oculto.

3 Robustez e Proteções para Nível Gratuito

Uma contribuição técnica relevante do repositório é a sua engenharia de robustez contra as limitações estritas de uso da API gratuita do Google AI Studio (especialmente com o modelo `gemini-3.1-flash-lite`):

- **Limitador de Taxa RPM:** Garante um intervalo mínimo entre requisições para respeitar o limite de 15 requisições por minuto (RPM).
- **Guarda de Tokens por Minuto (TPM):** Minimiza a transmissão de dados utilizando representações compactas para as grades e prompts, mantendo o tráfego abaixo de 250K TPM.

- **Guarda de Cota Diária (RPD):** Rastreia as chamadas acumuladas com um limite estrito de segurança de 1.450 requisições diárias (evitando o bloqueio rígido de 1.500 RPD).
- **Salvamento Incremental (Checkpoint & Resume):** O progresso do experimento é salvo no disco após cada tarefa individual concluída. Se houver interrupções por falta de cota ou falha de conexão, a execução pode ser retomada sem perda de trabalho. Backups de segurança são gerados a cada 5 tarefas.

4 Configuração Experimental

A execução de testes e benchmark é parametrizada de maneira flexível. O arquivo principal `main.py` permite definir o provedor de LLM (`gemini` ou `groq`), o modelo de destino (`MODEL_NAME`), o limite de tempo (`TIMEOUT_SECONDS`) e o atraso entre requisições.

O repositório suporta a criação de um pool de modelos distribuídos (`LLM_POOL`) para alternar requisições em *round-robin* entre diferentes provedores e chaves de API, distribuindo a carga de forma eficiente e contornando limites individuais de taxa.

5 Conclusão

O repositório `arc-agi-1` fornece uma infraestrutura experimental sólida para o estudo de síntese indutiva de programas sob restrições realistas de recursos. Ao formalizar a comparação entre o aprendizado 1-Shot simples e o refinamento estruturado guiado por feedback semântico (CEGIS), o framework pavimenta o caminho para investigações mais detalhadas sobre as capacidades de raciocínio de LLMs compactos.

Agradecimentos

Agradecemos aos contribuidores Matheus Machado Cezar e Izadora Candotti pelo desenvolvimento do repositório e disponibilização pública do código sob licença aberta para fins de pesquisa.

References

- Chollet, F. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*.
- da Costa Cieslak, F. L.; de Oliveira, I. C.; and Machado, M. 2026. ARC-AGI-1: Baseline vs CEGIS Comparative Experiment. <https://github.com/Jubarte27/arc-agi-1>. Acessado: 2026-08-29.